

SparkInject

SparkInject is a perpetual-license Sparkplug-to-cloud injection module, landing estate data in Snowflake or Databricks with store-and-forward and exactly-once semantics.

Perpetual license Ignition 8.1.19+ and 8.3 Runs on: Host

Contents

1. [1. Overview](#)
2. [2. What You Need](#)
3. [3. Install](#)
4. [4. License](#)
5. [5. Configure](#)
6. [6. Operate](#)
7. [7. Verify](#)
8. [8. Troubleshoot](#)
9. [9. Reference](#)
10. [10. Support](#)

1. Overview

SparkInject is a perpetual-license Sparkplug-to-cloud injection module for Ignition. It consumes `spBv1.0/#` as an ordinary MQTT client, decodes it with the same codec the SparkBridge Host uses, and lands it in Snowflake or Databricks with store-and-forward across a cloud outage and exactly-once semantics across a restart. The default sink writes local JSONL files so the pipeline can be proved end to end before a warehouse is provisioned.

The MQTT ingest thread only decodes and does non-blocking offers; a stalled cloud destination can never back-pressure the local MQTT dispatch loop. A bounded queue and disk buffer absorb the outage instead.

2. What You Need

- A SparkBridge Host gateway.
- A Snowflake or Databricks destination, provisioned before switching off the default JSONL sink.
- For Snowflake: a role and a key-pair user with `USAGE` and `INSERT`, `SELECT` grants, created from the module's generated DDL.

3. Install

1. SparkInject carries its own license. Install the signed module through Config > Modules > Install or Upgrade a Module; no build step is required.
2. Set `sparkinject.enabled=true`; it is off by default.
3. Leave `sparkinject.sink=jsonl` (the default) to prove the pipeline end to end before provisioning a warehouse.

4. License

Perpetual license. See the README in the bundle for purchasing. Standard suite key behavior applies: the file lives at `<data>/sparkbridge-license.key` and is re-checked every 30 seconds.

5. Configure

Before switching `sparkinject.sink=snowpipe`, run the generated DDL at `docs/sparkinject-schema.sql` to create the database, schema, and one table per mapped group, plus the role and key-pair user. The module cannot create tables itself.

<code>sparkinject.enabled</code>	false by default.
<code>sparkinject.sink</code>	jsonl (default) or snowpipe; databricks for that destination.
<code>sparkinject.mapping.file</code>	default <code>data/sparkinject-mapping.properties</code> , group-to-table routing.
<code>sparkinject.mapping.requireRule</code>	false by default; true refuses an unmapped group instead of deriving a table name.
<code>sparkinject.secrets.file</code>	default <code>data/sparkinject-secrets.properties</code> , mode 600, never on the JVM command line.
<code>sparkinject.broker.uri</code>	default <code>tcp://127.0.0.1:1883</code> .
<code>sparkinject.snowflake.privateKey</code>	secrets-file key; PEM PKCS#8 private key for the Snowflake key-pair user.
<code>sparkinject.status.token</code>	secrets-file key; bearer token for <code>status.json</code> , unset means loopback-only.

The landed table shape is one narrow EAV table per Sparkplug group, with columns for node key, device, metric, source timestamp, numeric/string/boolean value, quality (192 good, or a stale/death-marker code) and the Sparkplug `is_historical` flag.

6. Operate

SparkInject ingests passively (no STATE, no Will) and batches into the configured sink through a bounded queue. A cloud outage spills to disk via the store-and-forward buffer and replays the missed suffix on reconnect, using an offset-token discipline so Snowflake is told exactly what it has not received rather than replaying everything on disk.

7. Verify

Check `GET /data/io.sparkinject.snowflake/status.json`. A lane in state `BLOCKED` always carries a non-null problem field with pasteable remediation, for example a missing destination table. `inject.buffered` reports bytes currently held on disk and is the number to alarm on.

8. Troubleshoot

Symptom	Cause	Fix
No data landing	<code>sparkinject.enabled</code> is false (default)	Set it to true
Lane state <code>BLOCKED</code>	Destination table or schema does not exist yet	Read <code>status.json</code> 's problem field and run the missing DDL from <code>docs/sparkinject-schema.sql</code>
<code>inject.buffered</code> climbing steadily	Cloud destination unreachable or credentials invalid	Check Snowflake/Databricks connectivity and the key in <code>sparkinject.secrets.file</code>

Symptom	Cause	Fix
Unmapped group silently gets a derived table name	<code>sparkinject.mapping.requireRule</code> is false (default)	Set it to true to refuse unmapped groups instead, or add the mapping

9. Reference

- Status route: `GET /data/io.sparkinject.snowflake/status.json`.
- Secrets file default: `data/sparkinject-secrets.properties` (must be mode 600).
- Mapping file default: `data/sparkinject-mapping.properties`.
- Generated DDL: `docs/sparkinject-schema.sql` in the module bundle.

10. Support

[SparkInject in the docs](#) · [SparkInject product page](#) · [Contact](#) · [GitHub releases](#)